Real-World Applications of Stacks as a Data Structure

Assignment: Data Structures & System Implementation **Repository:** github.com/lk-chris/Weather-App

1. WHAT IS A STACK & WHY IS IT NECESSARY?

A **Stack** is a linear data structure that operates on the **Last-In, First-Out (LIFO)** principle: whatever was placed into the stack most recently is always the first element removed, just like a stack of plates or trays. Adding an element is called `push` (placed at the top), and removing an element is called pop (retrieved from the top). Both operations, along with inspecting the top element with peek, execute in instantaneous *O(1)* constant time.

Why Stacks are Necessary in Computing:

- **Execution Context & Function Calls (The Call Stack):** When programs run nested or recursive functions, the computer must remember where to return when each function finishes. A stack records each active function frame on top and pops it when the function completes, keeping memory execution orderly.
- **Reversible State Management (Undo / Redo & History):** Applications frequently require users to step backward through actions. By pushing each action or screen state onto a stack, the previous state is always preserved right beneath the top, making reversal a simple single-step pop.
- **Syntax Parsing & Delimiter Matching (Expression Validation):** Compilers and code linters check nested brackets like `{ [ ( ) ] }`. Opening symbols are pushed to the stack and popped when closing symbols match. Any leftover or mismatched symbol immediately catches syntax errors.
- **Algorithmic Backtracking (Depth-First Search):** When searching through mazes, decision trees, or network paths, a stack records the route explored so an algorithm can cleanly backtrack to the most recent decision point whenever it reaches a dead end.

2. REAL-WORLD APPLICATIONS OF STACKS

1. Browser Navigation History

Browsers push visited URLs onto a history stack; pressing the "Back" button pops the top page to reveal and restore the preceding one.

2. Text Editor Undo/Redo Engine

Every keystroke, paste, or deletion is pushed onto an undo stack. Pressing Ctrl+Z pops the latest operation to restore prior document state.

3. Runtime Call Stack & Recursion

Language runtimes allocate execution frames on a call stack to track function arguments, local variables, and return addresses safely.

4. Syntax Analysis & Expression Parsing

Compilers use stacks to convert infix arithmetic into postfix notation (Reverse Polish) and verify that code delimiters open and close properly.

3. SELECTED APPLICATION TO DEVELOP: WEATHER DASHBOARD & NAVIGATION HISTORY STACK

Application Overview & Stack Architecture

The application is a **Weather Forecast Dashboard** that integrates a **dual-stack navigation engine** (`historyStack` and `forwardStack`) to manage user location searches and screen transitions through the **LIFO (Last-In, First-Out)** principle. As users search and explore weather across various global cities, each location snapshot is **pushed** onto the navigation history stack. When users click the "Back" button, the active city is **popped** from the top of the stack and transferred to the forward stack, seamlessly restoring the previous city to the view in *O(1)* time.

- **Live Stack Visualization & Depth Counter:** Renders real-time stack depth (`Stack Depth: N`) and visibly marks the topmost active frame (`TOP / PEEK (Active View)`), demonstrating true LIFO state behavior.
- **Push Operation (City Search & Visit):** Captures weather metrics for newly searched locations, pushing a new state frame to the top of the history stack and resetting the forward stack.
- **Pop Operation (Back Navigation):** Pops the active city off the top of the stack and reveals the previous location with zero redundant API calls, updating the active viewport instantly.